

第1章 绪论

主讲教师：黄绍辉



目前，计算机已深入到社会生活的各个领域，其应用已不再仅仅局限于科学计算，而更多的是用于控制，管理及数据处理等非数值计算领域。计算机是一门研究用计算机进行信息表示和处理的科学。这里面涉及到两个问题：信息的表示，信息的处理。

信息的表示和组织又直接关系到处理信息的程序的效率。随着应用问题的不断复杂，导致信息量剧增与信息范围的拓宽，使许多系统程序和应用程序的规模很大，结构又相当复杂。因此，必须分析待处理问题中的对象的特征及各对象之间存在的关系，这就是数据结构这门课所要研究的问题。



计算机求解问题的一般步骤

编写解决实际问题的程序的一般过程：

- 如何用数据形式描述问题?—即由问题抽象出一个适当的数学模型;
- 问题所涉及的数据量大小及数据之间的关系;
- 如何在计算机中存储数据及体现数据之间的关系?
- 处理问题时需要对数据作何种运算?
- 所编写的程序的性能是否良好?

上面所列举的问题基本上由数据结构这门课程来回答。

1.1 数据结构及其概念

《算法与数据结构》是计算机科学中的一门综合性专业基础课，是介于数学、计算机硬件、计算机软件三者之间的一门核心课程，不仅是一般程序设计的基础，而且是设计和实现编译程序、操作系统、数据库系统及其他系统程序和大型应用程序的重要基础。



1.1.1 数据结构的例子

例1：电话号码查询系统

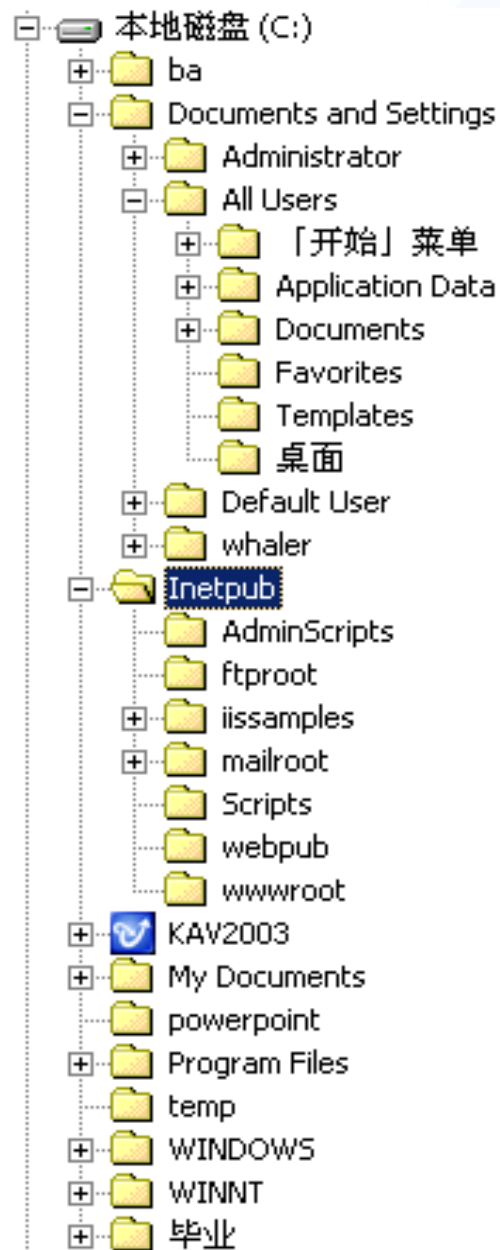
设有一个电话号码簿，它记录了N个人的名字和其相应的电话号码，假定按如下形式安排： (a_1, b_1) , (a_2, b_2) , ..., (a_n, b_n) ，其中 $a_i, b_i (i=1, 2 \dots n)$ 分别表示某人的名字和电话号码。本问题是一种典型的表格问题。如下表，数据与数据成简单的一对一的线性关系。

姓名	电话号码
陈海	13612345588
李四锋	13056112345
。 。 。	。 。 。

例2：磁盘目录文件系统

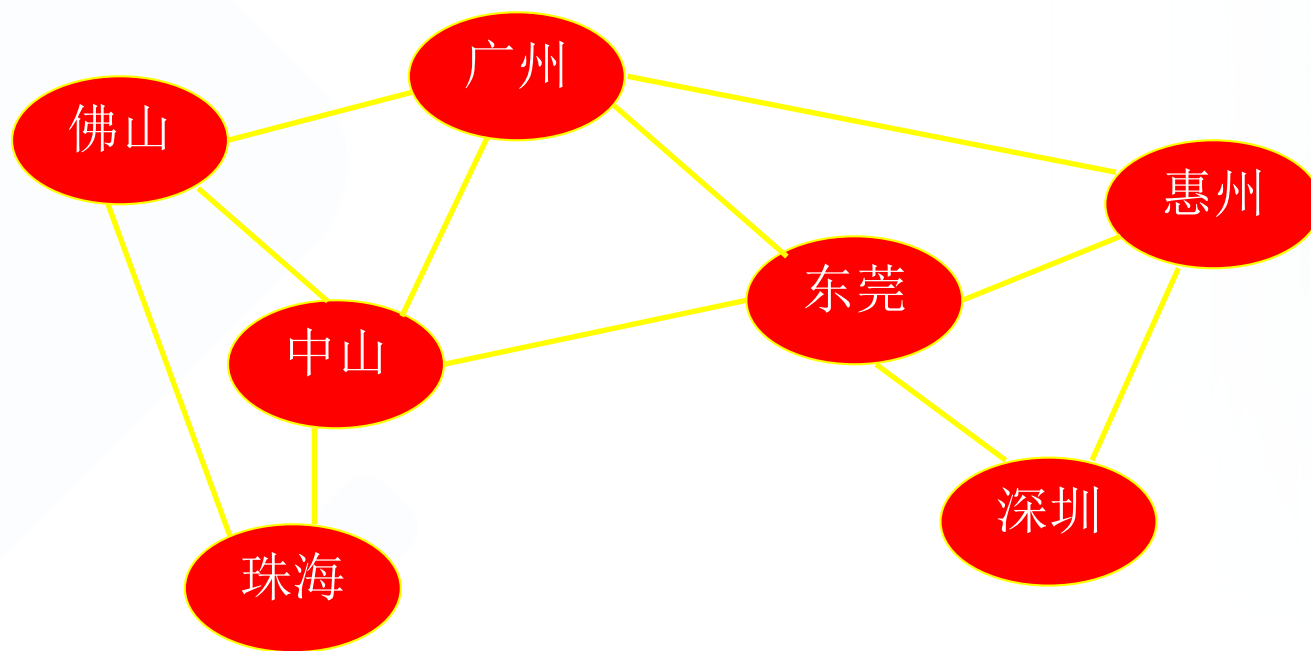
磁盘根目录下有很多子目录及文件，每个子目录里又可以包含多个子目录及文件，但每个子目录只有一个父目录，依此类推：

本问题是一种典型的树型结构问题，如图，数据与数据成一对多的关系，是一种典型的非线性关系结构——**树形结构**。



例3：交通网络图

从一个地方到另外一个地方可以有多条路径。本问题是一种典型的**网状结构**问题，数据与数据成多对多的关系，是一种非线性关系结构。



1.2 基本概念和术语

数据(Data)：是客观事物的符号表示。在计算机科学中指的是所有能输入到计算机中并被计算机程序处理的符号的总称。

数据元素(Data Element)：是数据的基本单位，在程序中通常作为一个整体来进行考虑和处理。

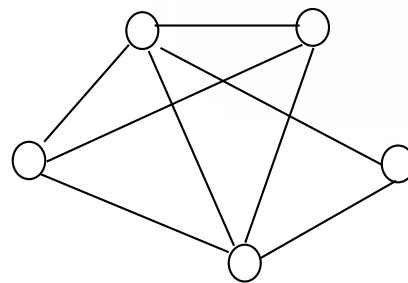
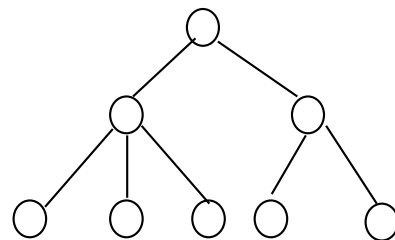
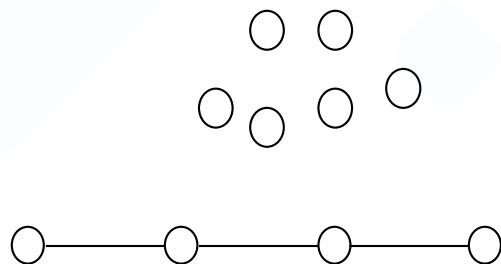
一个数据元素可由若干个**数据项(Data Item)**组成。数据项是数据的不可分割的最小单位。数据项是对客观事物某一方面特性的数据描述。

数据对象(Data Object)：是性质相同的数据元素的集合，是数据的一个子集。如字符集合 $C=\{'A', 'B', 'C', \dots\}$ 。



数据结构(Data Structure): 是指相互之间具有(存在)一定联系(关系)的数据元素的集合。元素之间的相互联系(关系)称为**逻辑结构**。数据元素之间的逻辑结构有四种基本类型, 如图所示。

- ① **集合**: 结构中的数据元素除了“同属于一个集合”外, 没有其它关系。
- ② **线性结构**: 结构中的数据元素之间存在一对一的关系。
- ③ **树型结构**: 结构中的数据元素之间存在一对多的关系。
- ④ **图状结构或网状结构**: 结构中的数据元素之间存在多对多的关系。



数据结构的形式定义

➤ 数据结构的形式定义是一个二元组：

$$\text{Data_Structure} = (D, S)$$

其中：D是数据元素的有限集，S是D上关系的有限集。

例：设数据逻辑结构 $B = (K, R)$

$$K = \{k_1, k_2, \dots, k_9\}$$

$$R = \{ \langle k_1, k_3 \rangle, \langle k_1, k_8 \rangle, \langle k_2, k_3 \rangle, \langle k_2, k_4 \rangle, \langle k_2, k_5 \rangle, \langle k_3, k_9 \rangle, \langle k_5, k_6 \rangle, \langle k_8, k_9 \rangle, \langle k_9, k_7 \rangle, \langle k_4, k_7 \rangle, \langle k_4, k_6 \rangle \}$$

根据以上描述，很容易画出逻辑结构B的图示，并确定那些是起点，那些是终点。



数据元素之间的关系可以是元素之间代表某种含义的自然关系，也可以是为处理问题方便而人为定义的关系，这种自然或人为定义的“关系”称为数据元素之间的逻辑关系，相应的结构称为逻辑结构。

数据结构的存储方式

- ◆ 数据结构在计算机内存中的存储包括**数据元素的存储和元素之间的关系**的表示。
- ◆ 元素之间的关系在计算机中有两种不同的表示方法：顺序表示和非顺序表示。由此得出两种不同的存储结构：**顺序存储结构**和**链式存储结构**。
 - **顺序存储结构**：用数据元素在存储器中的相对位置来表示数据元素之间的逻辑结构(关系)。
 - **链式存储结构**：在每一个数据元素中增加一个存放另一个元素地址的指针(pointer)，用该指针来表示数据元素之间的逻辑结构(关系)。



例：设有数据集A={3.0,2.3,5.0,-8.5,11.0}，可以设计两种不同的存储结构。

- **顺序结构：数据元素存放的地址是连续的；**
- **链式结构：数据元素存放的地址是否连续没有要求。**

数据的逻辑结构和物理结构是密不可分的两个方面，一个算法的设计取决于所选定的逻辑结构，而算法的实现依赖于所采用的存储结构。

在C语言中，用一维数组表示顺序存储结构；用结构体类型表示链式存储结构。

数据结构的三个组成部分：

逻辑结构： 数据元素之间逻辑关系的描述

$$D_S = (D, S)$$

存储结构： 数据元素在计算机中的存储及其逻辑关系的表现称为数据的存储结构或物理结构。

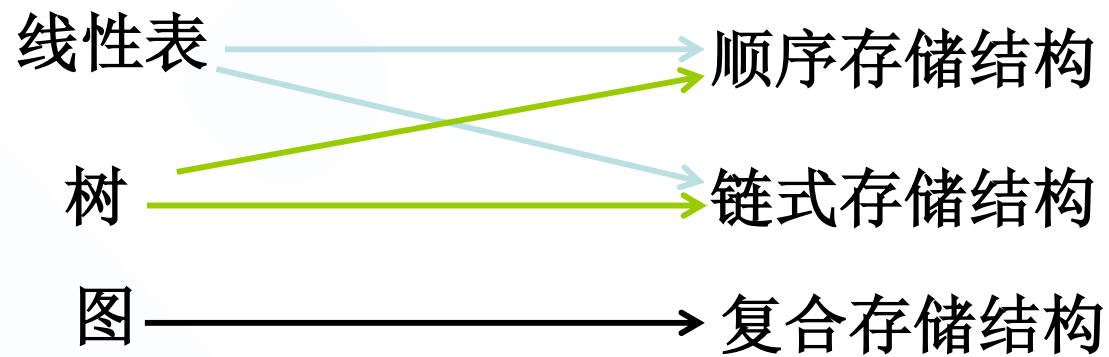
数据操作： 对数据要进行的运算。

本课程中将要讨论的三种逻辑结构及其采用的存储结构如下图所示。

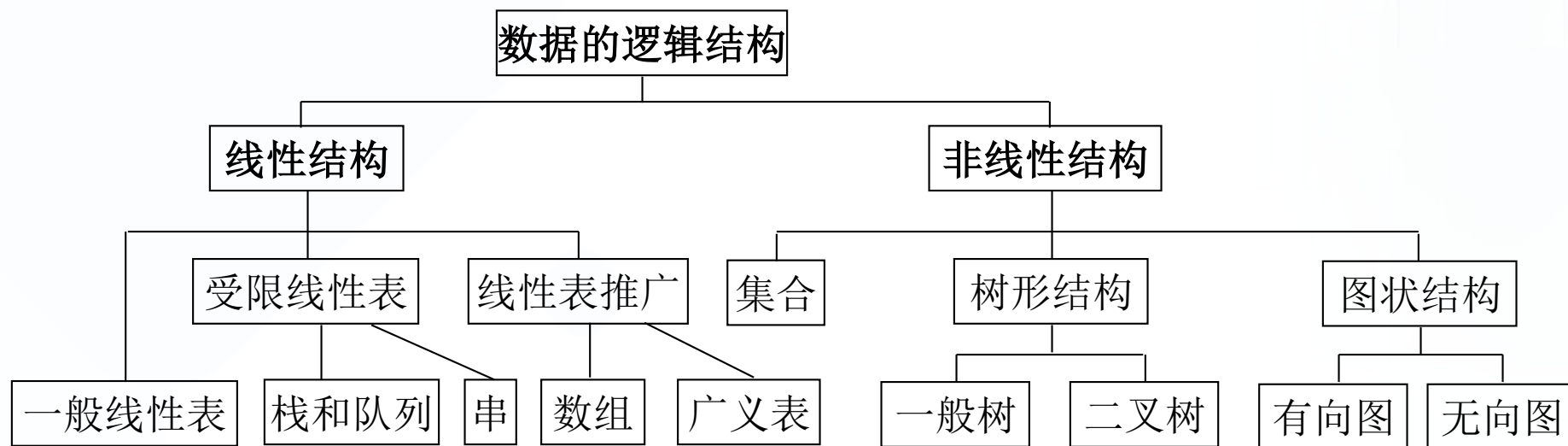


逻辑结构

物理结构



逻辑结构与所采用的存储结构



数据逻辑结构层次关系图

数据类型

- ◆ **数据类型(Data Type)**：指的是一个值的集合和定义在该值集上的一组操作的总称。
- ◆ 数据类型是和数据结构密切相关的一个概念。在C语言中数据类型有**基本类型**和**构造类型**。
- ◆ 数据结构不同于数据类型，也不同于数据对象，它不仅要描述数据类型的数据对象，而且要描述数据对象各元素之间的相互关系。



数据结构的运算

数据结构的主要运算包括：

- (1) 建立(Create)一个数据结构；**
- (2) 消除(Destroy)一个数据结构；**
- (3) 从一个数据结构中删除>Delete)一个数据元素；**
- (4) 把一个数据元素插入(Insert)到一个数据结构中；**
- (5) 对一个数据结构进行访问(Access)；**
- (6) 对一个数据结构(中的数据元素)进行修改(Modify)；**
- (7) 对一个数据结构进行排序(Sort)；**
- (8) 对一个数据结构进行查找(Search)。**



1.3 抽象数据类型的表示与实现

- ◆ **抽象数据类型**(**Abstract Data Type** , 简称**ADT**) : 是指一个数学模型以及定义在该模型上的一组操作。
- ◆ ADT的定义仅是一组逻辑特性描述 , 与其在计算机内的表示和实现无关。因此 , 不论ADT的内部结构如何变化 , 只要其数学特性不变 , 都不影响其外部使用。
- ◆ ADT的形式化定义是三元组 : $ADT = (D, S, P)$
其中 : D是**数据对象** , S是D上的**关系集** , P是对D的**基本操作集**。



ADT的一般定义形式是：

ADT <抽象数据类型名>{

数据对象： <数据对象的定义>

数据关系： <数据关系的定义>

基本操作： <基本操作的定义>

} ADT <抽象数据类型名>

其中数据对象和数据关系的定义用伪码描述。

基本操作的定义是：

<基本操作名>(<参数表>)

初始条件： <初始条件描述>

操作结果： <操作结果描述>

- 初始条件：描述操作执行之前数据结构和参数应满足的条件;若不满足，则操作失败，返回相应的出错信息。
- 操作结果：描述操作正常完成之后，数据结构的变化状况和 应返回的结果。

1.4 算法和算法分析

算法(Algorithm)：是对特定问题求解方法(步骤)的一种描述，是指令的有限序列，其中每一条指令表示一个或多个操作。

算法具有以下五个特性

- ① **有穷性**：一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
- ② **确定性**：算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- ③ **可行性**：一个算法必须是可行的。即算法描述的操作都可以通过已经实现的基本运算执行有限次来实现。



④ **输入**：一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。

⑤ **输出**：一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

一个算法可以用多种方法描述，主要有：使用自然语言描述；使用形式语言描述；使用计算机程序设计语言描述。

- ◆ **算法和程序是两个不同的概念。** 一个计算机程序是对一个算法使用某种程序设计语言的具体实现。算法必须可终止，意味着不是所有的计算机程序都是算法。
- ◆ 在本门课程的学习、作业练习、上机实践等环节，算法都用C语言来描述。在上机实践时，为了检查算法是否正确，应编写成完整的C++语言程序(不是C语言)。
- ◆ 建议熟悉C++的**标准模板库STL**（包括vector、stack、queue、map...等常用容器和sort等常用算法，会基本用法就行）。至于C++输入输出和面向对象的部分，可以忽略。

算法设计的要求

- ① **正确性(Correctness)**：算法应满足具体问题的需求。
- ② **可读性(Readability)**：算法应容易供人阅读和交流。
可读性好的算法有助于对算法的理解和修改。
- ③ **健壮性(Robustness)**：算法应具有容错处理。当输入非法或错误数据时，算法应能适当地作出反应或进行处理，而不会产生莫名其妙的输出结果。
- ④ **效率与存储量需求**：效率指的是算法执行的时间；存储量需求指算法执行过程中所需要的最大存储空间。一般地，这两者与问题的规模有关。



1.4.3 算法效率的度量

算法执行时间需通过依据该算法编制的程序在计算机上运行所消耗的时间来度量。其方法通常有两种：

1. 事后统计：计算机内部进行执行时间和实际占用空间的统计。

缺陷是：必须先运行依据算法编制的程序；依赖软硬件环境，容易掩盖算法本身的优劣；没有实际价值。

2. 事前分析：求出该算法的一个时间界限函数。



与此相关的因素有：

- 依据算法选用何种策略；
- 问题的规模；
- 程序设计的语言；
- 编译程序所产生的机器代码的质量；
- 机器执行指令的速度；

撇开软硬件等有关部门因素，可以认为一个特定算法“**运行工作量**”的大小，只依赖于问题的规模（通常用 n 表示），或者说，它是**问题规模的函数**。

算法分析应用举例

- ◆ 算法中**基本操作重复执行的次数**是问题规模 n 的某个函数，其时间量度记作 $T(n)=O(f(n))$ ，称作算法的渐近时间复杂度(**Asymptotic Time complexity**)，简称**时间复杂度**。
- ◆ 一般地，常用**最深层循环内**的语句中的原操作的**执行频度**(重复执行的次数)来表示。
- ◆ "O"的定义：若 $f(n)$ 是正整数 n 的一个函数，则 $O(f(n))$ 表示 $\exists M \geq 0$ ，使得当 $n \geq n_0$ 时， $|f(n)| \leq M |f(n_0)|$ 。

表示**时间复杂度**的阶有：

$O(1)$ ：常量时间阶

$O(n)$ ：线性时间阶

$O(\log n)$ ：对数时间阶

$O(n \log n)$ ：线性对数时间阶

$O(n^k)$ ： $k \geq 2$ ， k 次方时间阶



例 1 两个n阶方阵的乘法

```
for(i=1; i<=n; ++i)
    for(j=1; j<=n; ++j)
        { c[i][j]=0 ;
          for(k=1; k<=n; ++k)
              c[i][j] += a[i][k]*b[k][j] ; }
```

由于是一个三重循环，每个循环从1到n，则总次数为：
 $n \times n \times n = n^3$ 时间复杂度为 $T(n) = O(n^3)$

例 2 {++x; s=0 ;}

将x自增看成是基本操作，则语句频度为1，即时间复杂度为 $O(1)$ 。如果将s=0也看成是基本操作，则语句频度为2，其时间复杂度仍为 $O(1)$ ，即常量阶。

**例 3 `for(i=1; i<=n; ++i)`
 `{ ++x; s+=x ; }`**

语句频度为： $2n$ ，其时间复杂度为： $O(n)$ ，即为线性阶。

**例 4 `for(i=1; i<=n; ++i)`
 `for(j=1; j<=n; ++j)`
 `{ ++x; s+=x ; }`**

语句频度为： $2n^2$ ，其时间复杂度为： $O(n^2)$ ，即为平方阶。

定理：若 $A(n)=a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$ 是一个 m 次多项式，则 $A(n)=O(n^m)$

例 5 `for(i=2;i<=n;++i)`
 `for(j=2;j<=i-1;++j)`
 `{ ++x; a[i,j]=x; }`

语句频度为： $1+2+3+\dots+n-2=(1+n-2) \times (n-2)/2$
 $= (n-1)(n-2)/2 = n^2 - 3n + 2$

$\therefore$ 时间复杂度为 $O(n^2)$ ，即此算法的时间复杂度为平方阶。

— 一个算法时间为 $O(1)$ 的算法，它的基本运算执行的次数是固定的。因此，总的时间由一个常数（即零次多项式）来限界。而一个时间为 $O(n^2)$ 的算法则由一个二次多项式来限界。

以下六种计算算法时间的多项式是最常用的。其关系为：

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

– 指数时间的关系为：

$$O(2^n) < O(n!) < O(n^n)$$

当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊。因此，只要有人能将现有指数时间算法中的任何一个算法化简为多项式时间算法，那就取得了一个伟大的成就。

– 有的情况下，算法中基本操作重复执行的次数还随问题的输入数据集不同而不同。

例1：素数的判断算法。

```
void prime( int n)
```

```
/* n是一个正整数 */
```

```
{ int i=2 ;
```

```
while ( (n%i)!=0 && i*1.0< sqrt(n) ) i++ ;
```

```
if (i*1.0>sqrt(n) )
```

```
    printf( "%d 是一个素数\n" , n) ;
```

```
else
```

```
    printf( "%d 不是一个素数\n" , n) ;
```

```
}
```

嵌套的最深层语句是 $i++$ ；其频度由条件 $(n \% i) \neq 0 \ \&\& \ i * 1.0 < \sqrt{n}$ 决定，时间复杂度 $O(n^{1/2})$ 。

例2：冒泡排序法。

```
void bubble_sort(int a[] , int n)
```

```
{  
    for (i=n-1; change=TURE; i>1 && change; --i)  
        change=FALSE;  
        for (j=0; j<i; ++j)  
            if (a[j]>a[j+1])  
                { a[j]  $\longleftrightarrow$  a[j+1] ; change=TURE ; }  
}
```

- 最好情况：0次
- 最坏情况： $1+2+3+\cdots+n-1=n(n-1)/2$
- 平均时间复杂度为： $O(n^2)$

1.4.4 算法的存储空间需求

空间复杂度(Space complexity)：是指算法编写成程序后，在计算机中运行时所需存储空间大小的度量。记作： $S(n)=O(f(n))$ 其中： n 为问题的规模(或大小)

该存储空间一般包括三个方面：

- 指令常数变量所占用的存储空间;
- 输入数据所占用的存储空间;
- 辅助(存储)空间。

一般地，算法的**空间复杂度**指的是**辅助空间**。

- 一维数组 $a[n]$ ：空间复杂度 $O(n)$
- 二维数组 $a[n][m]$ ：空间复杂度 $O(n*m)$



结束

谢谢

